

Guía de Estudio — Tema 11: Expresiones y Estructuras de Control

Curso: Laboratorio de Programación y Lenguajes · UNTDF IDEI 2026 Módulo VIII · Semana 11 · Tema 11 Duración de la clase: 180 minutos (constraint absoluto) Lenguaje principal: TypeScript · Contrastes: Rust, Kotlin, Go, Python, C, Scheme Bibliografía principal: Sebesta, *Concepts of Programming Languages*, caps. 7–8 Bibliografía auxiliar: Gabbrielli & Martini, *Programming Languages: Principles and Paradigms*, cap. 6; Louden, *Programming Languages: Principles and Practice*, cap. 8 Baseline de contenido: `clase_dada.txt` (859 líneas) · Filminas F-00 a F-44 (45 filminas) Agente: Dra. Sofía (study-guide-writer) · Fecha: 2026-06-28

1. Introducción al tema

¿Por qué este tema importa? Porque la mayoría de los bugs silenciosos en código real no provienen de algoritmos mal concebidos, sino de **decisiones semánticas del lenguaje** que el programador no vio. Una expresión que parece equivaler a otra puede comportarse distinto por cinco razones: precedencia, asociatividad, orden de evaluación, efectos colaterales y coerciones. La pregunta de apertura de la clase lo resume:

¿Dos expresiones equivalentes siempre se comportan igual? [F-01]

```
let x = 1
const r = x++ + x
```

¿El problema está en la matemática o en la semántica del lenguaje? La respuesta es: en la semántica. Y este tema te da las herramientas para razonar sobre ella.

Este tema es la bisagra del módulo VIII: toma los tipos y variables del módulo VII (temas 09–10) y construye sobre ellos el control de flujo que toda programación imperativa usa. A su vez, prepara el terreno para el módulo XI (conurrencia), donde el `async/await` que veremos al final se profundiza.

Mapa conceptual [F-03] :

```
Expresiones → Evaluación → Asignación y efectos
  → Booleanos y short-circuit → Selección
  → Iteración → Saltos restringidos y mecanismos modernos
```

Cada bloque construye sobre el anterior. No se puede razonar sobre un `if` sin entender qué se evalúa primero; no se puede razonar sobre un bucle sin entender qué se mantiene verdadero en cada iteración.

2. Objetivos de aprendizaje

Al finalizar esta guía, el estudiante podrá [F-02] :

1. **Definir** formalmente expresión, sentencia y contexto de evaluación.
2. **Aplicar** precedencia, asociatividad y orden de evaluación de operandos, con y sin efectos colaterales.
3. **Diferenciar** la semántica de short-circuit versus evaluación estricta y justificar su uso por corrección, no solo por eficiencia.
4. **Analizar** la asignación como sentencia y como expresión, incluyendo patrones de bug históricos.
5. **Evaluar** el impacto de coerciones y conversiones en legibilidad, seguridad y verificabilidad.
6. **Comparar** selección simple, múltiple y anidada según criterios de mantenibilidad y acoplamiento.
7. **Razonar** sobre iteración con invariantes, terminación y mecanismos de escape.
8. **Explicar** iteradores y generadores como abstracciones de control de flujo.
9. **Disecar** anti-patrones de control (goto indiscriminado, cascadas no normalizadas, efectos ocultos).
10. **Integrar** teoría y práctica en lectura crítica de código real.

Fuente: `disenio.md` — Objetivos de aprendizaje (versión extendida).

3. Conceptos previos necesarios

Esta guía asume que ya manejas los contenidos de los temas 09 y 10. No los re-explicamos, pero los usamos activamente:

Concepto previo	Tema	Qué necesitás recordar
Variables y mutabilidad	09	Qué es el estado de un programa; diferencia entre <code>let</code> y <code>const</code> .
Tipos de datos estáticos vs. dinámicos	10	Cómo el sistema de tipos detecta (o no) errores antes de ejecución.
Tipado fuerte vs. débil	10	Por qué Haskell/Rust minimizan coerciones y TypeScript/Python las permiten.
Funciones puras vs. con efecto	10	La noción de transparencia referencial: misma entrada → misma salida.

Si algún punto de esta lista te resulta difuso, conviene repasar la guía del tema 10 antes de continuar: el razonamiento sobre efectos colaterales (sección 4.1) depende directamente de entender qué es una función pura.

4. Desarrollo teórico

El desarrollo sigue los cinco bloques de la clase: A (expresiones), B (booleanos y short-circuit), C (selección), D (iteración) y E (cierre con `async/await`). Cada sección integra el concepto, una cita bibliográfica verificada en la base de conocimiento, el código textual de la clase y la referencia a la filmina correspondiente.

4.1 Bloque A — Fundamentos de expresiones y semántica [F-04 a F-16]

4.1.1 Expresión y sentencia

Una **expresión** es una construcción sintáctica que se evalúa y produce un valor. Una **sentencia** es una unidad ejecutable del programa que controla la ejecución o provoca un efecto. En lenguajes imperativos, muchas sentencias modifican estado. [F-04]

La distinción no es académica: en TypeScript una asignación **también es una expresión**, es decir, produce un valor además de causar un efecto.

```
let x = 0
let y = (x = 5) // y vale 5: la asignación devuelve el valor asignado
```

“Assignment is the basic command for the ‘modification of a variable’, that is, for changing the value (or the content) associated with a name. It is the most elementary command in every imperative language; still, there are various subtleties to be taken into account.” — [Gabbrielli & Martini, *Programming Languages: Principles and Paradigms*, cap. 6, p. 136]

Por qué importa [F-05] : si una expresión solo calcula, es más fácil razonar sobre ella. Si además muta estado, aparece riesgo semántico: la misma sintaxis mezcla cálculo y efecto.

```
let total = 0
function sumar(x: number): number {
  total += x // efecto colateral: muta estado global
  return total
}
```

Haskell favorece expresiones puras (sin efectos). TypeScript permite ambos estilos. Rust controla más estrictamente la mutabilidad. La consecuencia práctica: una función con efectos colaterales **rompe la transparencia referencial** — llamarla dos veces con los mismos argumentos puede dar resultados distintos.

4.1.2 AST y parseo

El **árbol sintáctico abstracto (AST)** es la representación intermedia que fija la semántica de una expresión. La precedencia de operadores determina la estructura del árbol; la asociatividad resuelve empates entre operadores de igual precedencia. [F-06]

⚠ Punto crítico: el AST captura **estructura**, no **evaluación**. Parseo y evaluación son fases distintas. El árbol no garantiza el orden temporal en que se evalúan los operandos.

```
const r = a + b * c // se parsea como a + (b * c) porque * tiene mayor precedencia
```

El operador con mayor precedencia queda más profundo en el árbol y se evalúa primero en términos de estructura — pero el orden temporal de evaluación de los operandos **a**, **b**, **c** lo define el lenguaje, no el AST.

“When an expression includes two different operators, for example, $x + y * z$, one obvious semantic issue is the order of evaluation of the two operators (...). This semantic question can be answered by assigning different precedence to the operators.” — [Sebesta, *Concepts of Programming Languages*, cap. 7, §7.2, p. 325]

4.1.3 Precedencia de operadores

La **precedencia** determina la agrupación sintáctica. Multiplicación precede a suma. Los paréntesis expresan intención. La tabla de precedencia es una **decisión de diseño** del lenguaje, no una ley universal. [F-07]

```
const r = a + b << c // ¿se suma antes o se desplaza antes? Depende del lenguaje.
```

La precedencia no es universal [F-08]: las reglas aritméticas suelen coincidir entre lenguajes, pero las lógicas, bit a bit y de asignación varían.

```
const a = 2 + 3 * 4 // 14 - * precede a +
const b = (2 + 3) * 4 // 20 - paréntesis fuerza la agrupación
```

TypeScript, Java y Kotlin tienen reglas similares. Python agrega ****** como operador de exponenciación. Scheme evita el problema usando notación prefija: no hay ambigüedad porque el operador va primero.

“If the addition and subtraction operators have the same level of precedence, as they do in programming languages, the precedence rules say nothing about the order of evaluation of the operators in this expression.” — [Sebesta, *Concepts of Programming Languages*, cap. 7, §7.2, p. 325]

4.1.4 Asociatividad

Cuando dos operadores tienen la misma precedencia, la **asociatividad** resuelve el empate: ¿izquierda o derecha? [F-09]

- **Asociatividad izquierda:** mayoría de los operadores aritméticos (**+**, **-**, *****, **/**).
- **Asociatividad derecha:** exponenciación, asignación en C/Java, operador condicional ternario.

```
const r = 10 - 3 - 2 // (10 - 3) - 2 = 5 - asociatividad izquierda
```

```
(- (- 10 3) 2) ; Scheme: la notación prefija elimina la ambigüedad
```

Caso crítico: `a = b = c = 5` en C evalúa de derecha a izquierda (asignación es asociativa derecha) — primero asigna 5 a `c`, luego el resultado a `b`, luego a `a`.

“When an expression includes two operators that have the same precedence (as `*` and `/` usually have)—for example, `A / B * C`—a semantic rule is required to specify which should have precedence. This rule is named associativity.” — [Sebesta, *Concepts of Programming Languages*, cap. 7, §7.2, p. 325]

Paréntesis como decisión semántica [F-10] : parentizar no es redundante si mejora la lectura. Si una expresión exige recordar demasiadas reglas, se usan paréntesis.

```
const habilitado =  
  (usuario.activo && usuario.emailVerificado) || usuario.esAdmin
```

Los paréntesis aquí comunican intención: primero la conjunción, luego la disyunción. Es **redundancia productiva**.

“It is useful to include parentheses when in doubt about precedence and associativity.” — [Gabbrielli & Martini, *Programming Languages: Principles and Paradigms*, cap. 6, p. 136]

4.1.5 Orden de evaluación de operandos

Este es el punto donde la semántica operacional entra en juego. **Parseo ≠ orden de evaluación temporal.** [F-11]

- El AST indica estructura.
- El lenguaje define —o deja sin definir— el orden temporal.
- Con funciones puras, el orden puede no importar.
- Con efectos colaterales, importa mucho.

“The semantics of an expression is determined in large part by the order of evaluation of operators. The associativity and precedence rules for operators in the expressions of a language determine the order of operator evaluation in those expressions. Operand evaluation order is important if functions (...) have side effects.” — [Sebesta, *Concepts of Programming Languages*, cap. 7, §7.2, p. 325]

4.1.6 Efectos colaterales en expresiones

TypeScript define más orden que C, pero no elimina el problema de diseño. [F-12]

```
let i = 1  
const r = i++ + ++i // el resultado depende del orden garantizado por el lenguaje
```

```
/* C/C++: algunos casos son indefinidos o no especificados según el caso */  
i = i++ + ++i;
```

```
// Rust: no existe i++ — decisión de diseño que elimina la clase entera de bugs
i += 1;
```

Regla de diseño: en TypeScript, la expresión puede estar definida pero seguir siendo ilegible. Definido $\neq$ correcto. Evitar expresiones que mezclen cálculo y mutación en la misma sentencia.

4.1.7 Asignación como expresión

La asignación puede ser simultáneamente sentencia y expresión. Esto habilita patrones idiomáticos pero también el bug clásico. [F-13]

```
/* C idiomático: getchar() asigna y su valor se compara con EOF */
while ((c = getchar()) != EOF) {
    procesar(c);
}
```

```
// TypeScript
let x = 0
let y = (x = 5) + 3 // y = 8
```

```
// Kotlin: no se usa asignación como expresión de valor
x = 5
```

Riesgo: los lenguajes que admiten *assignment expressions* en condiciones habilitan el bug clásico `if (x = 0)` — una asignación accidental donde se pretendía una comparación. C lo permite; los compiladores modernos emiten *warning* con `-Wall`. Rust, Swift y Kotlin **restringen el patrón por diseño**.

4.1.8 Prevención de bugs de asignación [F-14]

- **Yoda conditions:** `if (0 == x)` → una asignación accidental `if (0 = x)` falla al compilar.
- **Warnings del compilador:** `-Wparentheses`, `-Wall` en GCC/Clang.
- **Diseño de lenguaje:** Rust, Swift y Kotlin eliminan el problema por diseño.
- **Linters:** ESLint/TSLint detectan automáticamente este patrón.

4.1.9 Conversión y coerción [F-15]

Mecanismo	Control	Riesgo	Ejemplo
Conversión explícita (cast)	Programador	Bajo	<code>int(3.7) → 3</code>
Coerción implícita widening	Lenguaje	Medio	<code>int → float</code> automático
Coerción implícita narrowing	Lenguaje	Alto	<code>float → int</code> (pérdida de datos)
Sobrecarga de operadores	Lenguaje/Prog.	Variable	<code>+</code> en strings y enteros

Principio de Sebesta: la coerción implícita puede enmascarar errores de tipo que el sistema de tipos debería detectar. Lenguajes con tipado estático fuerte (Haskell, Rust) minimizan las coerciones implícitas.

“In a clear departure from C++, Java and C# allow mixed-mode assignment only if the required coercion is widening. So, an int value can be assigned to a float variable, but not vice versa. Disallowing half of the possible mixed-mode assignments is a simple but effective way to increase the reliability.” — [Sebesta, *Concepts of Programming Languages*, cap. 7, §7.8, p. 325]

4.1.10 Control conceptual — Bloque A [F-16]

Antes de avanzar, verificá tu comprensión con estas tres preguntas:

- 1. ¿Cuál es la diferencia entre precedencia de operadores y orden de evaluación de operandos?
- 2. ¿Cuándo una *assignment expression* en un `if` se convierte en bug?
- 3. Da un ejemplo donde la coerción implícita produce un resultado diferente al esperado.

Las respuestas están en la autoevaluación de la sección 7 (preguntas 1, 4 y 5).

4.2 Bloque B — Booleanos, short-circuit y seguridad semántica [F-17 a F-25]

4.2.1 Álgebra booleana aplicada

En código real, `&&` y `||` no son solo tablas de verdad: **controlan qué sub-expresiones se evalúan**. La conjunción y disyunción permiten codificar precondiciones de forma declarativa. [F-17]

Sebesta distingue operadores lógicos con y sin short-circuit: tienen semánticas distintas. El short-circuit es **semántica perezosa** de operadores lógicos.

4.2.2 Truthiness entre lenguajes

Qué cuenta como `true` varía entre lenguajes. No migrar intuiciones de truthiness de un lenguaje a otro sin validar la semántica local. [F-18]

Lenguaje	<code>0</code> es falsy	<code>""</code> es false	<code>null / None</code> false	Bool estricto
C	Sí	N/A	—	No (usa <code>int</code>)
Python	Sí	Sí	Sí	Sí, flexible
TypeScript	Sí	Sí	Sí	Sí, flexible
Java	No	No	Sí	Sí, estricto
Kotlin	No	No	Error de compilación	Sí, estricto

```
if (") { ... } // TypeScript: válido, falsy
```

```
if (") { ... } // Kotlin: error de compilación
```

Variación semántica referenciada en Sebesta §7.5–7.6 y Louden cap. 8.

4.2.3 Evaluación estricta vs. short-circuit

Evaluación estricta [F-19]: todos los operandos se evalúan siempre antes de aplicar el operador lógico. Es predecible y conveniente para análisis formal y optimizaciones del compilador. Pero puede ejecutar sub-expresiones inválidas innecesariamente (división por cero, acceso null). El Pascal original usaba `and` / `or` sin garantía de short-circuit.

Short-circuit [F-20]: la evaluación se detiene cuando el resultado ya es conocido.

- `p && q`: si `p` es **false**, `q` no se evalúa — el resultado ya es false.
- `p || q`: si `p` es **true**, `q` no se evalúa — el resultado ya es true.

Es una herramienta de **corrección semántica**, no solo de rendimiento. Permite usar la primera condición como guarda de seguridad de la segunda.

“The main reason for a short-circuit evaluation of Boolean operators is not efficiency, however. With full evaluation, certain common program schemata would be incorrect, or require baroque coding.” — [Gabbrielli & Martini, *Programming Languages: Principles and Paradigms*, cap. 6, p. 136]

“A short-circuit evaluation of an expression is one in which the result is determined without evaluating all of the operands and/or operators.” — [Sebesta, *Concepts of Programming Languages*, cap. 7, §7.6, p. 325]

Síntesis: la evaluación estricta favorece la **verificabilidad**; el short-circuit favorece la **corrección operativa**. No son opuestos absolutos: son trade-offs de diseño.

4.2.4 Patrón defensivo con short-circuit

El short-circuit como guarda de corrección: evitar división por cero y acceso null. [F-21]

```
// Guarda contra división por cero
if (x !== 0 && y / x > 2) {
  aprobar()
}

// Guarda contra acceso null
if (user !== null && user.isActive()) {
  procesar(user)
}
```

```
/* C */
if (p != NULL && p->value > 0) { ... }
```

Por qué funciona: si la primera condición falla, la segunda nunca se evalúa. Es una forma práctica de guarda operacional. ⚠️ **Invertir el orden destruye la guarda** y puede causar un *runtime error*: `y / x > 2 && x !== 0` evalúa la división antes de verificar que `x` no es cero.

“Short-circuit evaluation. The problem detailed in the previous point presents itself with particular clarity when evaluating Boolean expressions. For example, consider the following expression (in C syntax): `a == 0 || b / a > 2`.” — [Gabbrielli & Martini, *Programming Languages: Principles and Paradigms*, cap. 6, p. 136]

4.2.5 Side effects en booleanos: anti-patrón [F-22]

Mezclar predicados con efectos en operadores lógicos es un anti-patrón. El short-circuit convierte un efecto secundario en un **efecto condicional** → comportamiento inesperado.

```
// ANTI-PATRÓN: logAndMutate() puede no ejecutarse si isReady() es true
if (isReady() || logAndMutate()) {
  ejecutar()
}

// CORRECTO: el efecto ocurre siempre, independiente del predicado
const logged = logAndMutate()
if (isReady() || logged) {
  ejecutar()
}
```

Principio: separar predicados (sin efectos) de funciones con efectos colaterales. Si querés que un efecto sea condicional, sé explícito con un `if`; no lo escondas en un operador lógico.

4.2.6 Operadores lógicos: comparativa por lenguaje [F-23]

Lenguaje	AND lógico	OR lógico	AND bit	OR bit	NOT lógico
C/C++	<code>&&</code>	<code> </code>	<code>&</code>	<code> </code>	<code>!</code>
Java	<code>&&</code>	<code> </code>	<code>&</code>	<code> </code>	<code>!</code>
TypeScript	<code>&&</code>	<code> </code>	<code>&</code>	<code> </code>	<code>!</code>
Python	<code>and</code>	<code>or</code>	<code>&</code>	<code> </code>	<code>not</code>
Kotlin	<code>&&</code>	<code> </code>	<code>and</code>	<code>or</code>	<code>!</code>

⚠️ Los operadores bit a bit (`&`, `|`) **no cortocircuitan**: evalúan ambos operandos siempre. Solo los operadores lógicos (`&&`, `||`, `and`, `or`) tienen semántica de short-circuit.

4.2.7 Null safety con operadores lógicos [F-24]

TypeScript tiene operadores especializados para null que son formas de short-circuit:

```
// Optional chaining: corta en null/undefined
const ciudad = usuario?.direccion?.ciudad

// Nullish coalescing: default solo para null/undefined (no para 0 ni "")
const nombre = entrada ?? "sin nombre"

// Combinados: guarda completa de acceso y fallback
const zip = usuario?.direccion?.codigoPostal ?? "0000"
```

```
val ciudad = usuario?.direccion?.ciudad
val nombre = entrada ?: "sin nombre"
```

```
let ciudad = usuario
  .and_then(|u| u.direccion)
  .map(|d| d.ciudad);
```

Semántica: `?.` implementa short-circuit ante null o undefined. `??` es más estricto que `||`: no cortocircuita ante `0` o `""` (que pueden ser valores válidos), solo ante `null` / `undefined`.

4.2.8 Guard clauses [F-25]

Reducir anidamiento con cláusulas de guarda (*early return*):

```
// SIN guard clauses: anidamiento profundo
function procesar(x?: number) {
  if (x !== null) {
    if (x >= 0) { return `ok:${x}` }
  }
  return "inválido"
}

// CON guard clauses: flujo plano y legible
function procesar(x?: number) {
  if (x == null) return "faltante"
  if (x < 0) return "inválido"
  return `ok:${x}`
}
```

El flujo plano es más legible. Un *return* por guarda es preferible a un anidamiento profundo: reduce complejidad accidental sin cambiar la semántica.

4.3 Bloque C — Selección estructurada y decisiones de diseño [F-26 a F-33]

4.3.1 Programación estructurada y el debate sobre goto

`goto` permite saltos arbitrarios a cualquier punto del programa. Aumenta el poder de expresión local; reduce la trazabilidad y verificabilidad global. [F-26]

Dijkstra (1968): “*Go To Statement Considered Harmful*” — fundamento del movimiento estructurado. Las estructuras de control buscan control explícito: **entrada única, salida única, verificable**.

“The unconditional branch, or goto, has been part of most imperative languages. Its problems have been widely discussed and debated. The current consensus is that it should remain in most languages but that its dangers should be minimized through programming discipline.” — [Sebesta, *Concepts of Programming Languages*, cap. 8, p. 325]

“Despite its apparent simplicity and naturalness, the goto command has been at the centre of a considerable debate since the start of the 1970s (see, for example, a famous article by Dijkstra), between its supporters and its detractors, who in the end won the match.” — [Gabbrielli & Martini, *Programming Languages: Principles and Paradigms*, cap. 6, §6.4, p. 136]

El `goto` moderno restringido (C, C++) sobrevive para manejo de errores y salida de bucles anidados — uso justificado y acotado. Go tiene `goto` restringido:

```
package main

import "fmt"

func main() {
    i := 0
inicio:
    fmt.Println(i)
    i++
    if i < 3 {
        goto inicio
    }
}
```

`goto` está permitido en Go, pero restringido: permite saltar a una etiqueta dentro de la misma función, sin entrar ilegalmente en un bloque ni saltarse inicializaciones de variables.

Referencia bibliográfica: Dijkstra, E. W. (1968a). “Goto Statement Considered Harmful.” *Communications of the ACM*, Vol. 11, No. 3, pp. 147–149. — [Sebesta, *Concepts of Programming Languages*, bibliografía, p. 703]

4.3.2 If y else if: árbol de decisiones [F-27]

Propiedades del `if-else`:

- Ramas **mutuamente excluyentes**.
- Evaluación **en orden**.
- `else` como **caso no capturado**.
- El **orden de condiciones importa**.

```
if (score >= 9)    grado = "A"
else if (score >= 7) grado = "B"
else if (score >= 4) grado = "C"
else              grado = "D"
```

```
val grado =
    if (score >= 9) "A"
    else if (score >= 7) "B"
    else if (score >= 4) "C"
    else "D"
```

⚠ En Kotlin, `if` es una **expresión** que devuelve un valor. En TypeScript, `if` es una **sentencia**. Esta diferencia de diseño afecta cómo se escribe el código idiomático.

“The else-if version (the first) is the more readable of the two. Notice that this example is not easily simulated with a switch statement, because each selectable statement is chosen on the basis of a Boolean expression. Therefore, the else-if statement is not a redundant form of switch.” — [Sebesta, *Concepts of Programming Languages*, cap. 8, p. 325]

4.3.3 Switch: selección múltiple [F-28]

`switch` clásico discrimina por valor. C/JavaScript/TypeScript heredan la necesidad de `break`. Kotlin reemplaza `switch` por `when`. Rust usa `match` exhaustivo.

```
switch (token) {  
  case "INT": parseIntToken(); break  
  case "ID":  parseIntToken(); break  
  case "STR": parseIntToken(); break  
  default:   reportError()  
}
```

```
when (token) {  
  "INT" -> parseIntToken()  
  "ID"  -> parseIntToken()  
  "STR" -> parseIntToken()  
  else  -> reportError()  
}
```

El problema de confiabilidad aparece cuando hay **continuación implícita** de una rama a otra (*fallthrough*): olvidar un `break` hace que la ejecución caiga al siguiente `case`. Lenguajes modernos eliminan el *fallthrough* por diseño.

“A multiple selection statement is essentially an n-way branch to segments of code, where n is the number of selectable segments.” — [Sebesta, *Concepts of Programming Languages*, cap. 8, p. 325]

4.3.4 Pattern matching: evolución del control múltiple [F-29]

- `switch` clásico: discrimina por **valor** (escalar, enum).
- **Pattern matching**: discrimina por **estructura** del dato — forma, tipo y desestructuración.

Disponible en Haskell, Scala, Rust, Python 3.10+, Java 21+. En lenguajes con tipos algebraicos, pattern matching reemplaza la selección como estructura primaria.

4.3.5 Complejidad cognitiva del anidamiento [F-30]

Cada nivel de anidamiento extra multiplica los *paths* de ejecución posibles. Las ramas profundas elevan el riesgo de *paths* no testeados en revisión manual. La **complejidad ciclomática** mide el número de *paths* linealmente independientes. Guard clauses y *early return* reducen complejidad accidental sin cambiar la semántica.

Regla práctica: máximo 3 niveles de anidamiento antes de extraer una función o reestructurar el flujo.

4.3.6 Criterios para elegir estructura de selección [F-31]

Criterio	if-else	switch	Tabla de despacho
Pocas ramas (2-3)	óptimo	Posible	Sobreingeniería
Muchas ramas (>5)	Verboso	legible	extensible
Condiciones complejas	natural	Limitado	—
Cambio frecuente de casos	Costoso	Costoso	Open/Closed
Exhaustividad verificable	Manual	Sí (con default)	Manual

La elección no es estética: es una decisión de ingeniería de mantenimiento.

4.3.7 Despacho por tabla [F-32]

Separar lógica de control de los datos:

```
const handlers: Record<string, () => void> = {  
  INT: parseIntToken,  
  ID:  parseIdToken,  
  STR: parseStringToken,  
};  
  
(handlers[token] ?? reportError)();
```

Buscar en la tabla `handlers` la función asociada al valor de `token`; si no existe, usar `reportError`; luego ejecutar la función elegida.

Ventajas: agregar un caso = agregar una entrada, no se modifica lógica de control. Principio **Open/Closed** aplicado a estructuras de selección. Cada rama es testeable independientemente.

4.3.8 Code smells de selección [F-33]

- **Condiciones duplicadas** en múltiples ramas → violan DRY y ocultan semántica.
 - **Default que esconde errores** → enmascarar casos no manejados reduce trazabilidad.
 - **Predicados opacos con side effects** → mezclan responsabilidades de evaluación y acción.
 - **Cascadas largas sin dominio explícito** → señal de que falta una abstracción de datos.
-

4.4 Bloque D — Iteración, iteradores y generadores [F-34 a F-42]

4.4.1 Estructuras iterativas clásicas [F-34]

- **while** : evalúa condición al inicio → puede no ejecutarse si la condición es falsa de entrada.
- **do-while** : evalúa condición al final → garantiza al menos una ejecución.
- **for** : contador, condición y actualización en una línea → para iteración acotada.

En la mayoría de lenguajes modernos, **for** es **azúcar sintáctico** sobre **while**.

```
while (hayDatos())      leer()
do { leer() } while (hayDatos())
for (let i = 0; i < n; i++) procesar(i)
```

```
while (hayDatos())      leer()
for (i in 0 until n)    procesar(i)
```

4.4.2 Invariantes de bucle [F-35]

El **invariante** garantiza la corrección del bucle. Es una propiedad que debe mantenerse verdadera en cuatro momentos:

1. **Qué se mantiene verdadero**: la propiedad que define el invariante.
2. **Cómo se establece**: el invariante debe ser verdadero **antes** de entrar al bucle.
3. **Cómo se preserva**: el cuerpo del bucle debe mantener el invariante en **cada iteración**.
4. **Qué permite concluir al terminar**: invariante + negación de la condición → resultado correcto.

Los invariantes son la base de la **verificación formal** (Lógica de Hoare). No tenés que escribir el invariante en el código, pero tenerlo mental te ayuda a razonar sobre la corrección.

“The loop invariant must satisfy a number of requirements to be useful. First, the weakest precondition for the while loop must guarantee the truth of the loop invariant. In turn, the loop invariant must guarantee the truth of the post-condition upon loop termination.” — [Sebesta, *Concepts of Programming Languages*, cap. 3 (Verificación de programas / Lógica de Hoare), p. 25]

4.4.3 Terminación del bucle [F-36]

El bucle termina si hay **progreso medible** hacia la condición de parada.

- Definir una **función de ranking** (*loop variant*): entero acotado inferiormente que decrece en cada iteración.
- El bucle termina si el variant es **estrictamente decreciente** y acotado.
- Verificar **casos borde**: `n = 0`, colecciones vacías, centinela inalcanzable.
- Bucles con centinela: la terminación depende de que el centinela sea alcanzable en la entrada.

⚠ Si no encontrás un *variant*, es una señal de que el bucle podría no terminar. Hay que revisar la lógica.

4.4.4 Break y continue [F-37]

`break` y `continue` son **transferencias estructuradas**: afectan solo el bucle más cercano. No son `goto`: no saltan a cualquier punto, solo al cierre del bucle.

```
for (const x of xs) {  
  if (!esValido(x)) continue // salta al próximo elemento  
  if (x === objetivo) {  
    encontrado = true  
    break // sale del bucle  
  }  
  procesar(x)  
}
```

Java permite `break label` para salir de bucles anidados — transferencia estructurada restringida, similar al `goto` limitado.

“Jump into a loop. The last point which merits attention concerns the possibility of jumping into the middle of a for loop using a goto command. Most languages forbid such jumps for clear semantic reasons, while there are fewer restrictions on the possibility of using a goto for jumping out of a loop.” — [Gabbrielli & Martini, *Programming Languages: Principles and Paradigms*, cap. 6, p. 136]

4.4.5 Contador y centinela [F-38]

- **Contador**: control por límites explícitos; adecuado cuando la cota es conocida de antemano.
- **Centinela** (`while`): control por valor especial en el *stream*; evita evaluar longitud en cada iteración.

El centinela es útil en lectura incremental de *streams* o archivos: `while ((c = getchar()) != EOF)` en C. ⚠ Riesgo: si el valor centinela nunca aparece en la entrada, el bucle no termina.

4.4.6 Iteradores [F-39]

El iterador **desacopla** la colección del recorrido:

- La **colección** almacena los datos.
- El **iterador** encapsula el estado de recorrido y define el *traversal*.
- Habilita recorridos alternativos sobre la misma estructura sin duplicar su estado interno.
- El protocolo `Iterable/Iterator` (TypeScript, Java, Python) estandariza el contrato.

Los iteradores son la forma moderna de iterar sobre estructuras sin exponer su representación interna. El `for...of` consume el protocolo `Iterable/Iterator` por debajo.

4.4.7 Generadores: secuencias perezosas [F-40]

El generador **suspende** la ejecución en cada `yield` y la **reanuda** al siguiente `next()`. Permite trabajar con secuencias infinitas sin consumir memoria proporcional al tamaño. Cada `next()` reanuda la función exactamente donde se suspendió.

```
function* rango(inicio: number, fin: number) {  
  for (let i = inicio; i < fin; i++) yield i  
}
```

```
def rango(inicio, fin):  
    for i in range(inicio, fin):  
        yield i
```

“When yield is executed, the activation record of its generator is not destroyed. Moreover, the program location where the yield occurred is saved, so that a subsequent next on the same generator will resume the execution from where it was paused and with the environment and memory in which it was paused.” — [Gabbrielli & Martini, *Programming Languages: Principles and Paradigms*, cap. 6, p. 136]

“In Python, any method that contains a yield statement is called a generator, because it generates data one element at a time.” — [Sebesta, *Concepts of Programming Languages*, cap. 7, p. 325]

4.4.8 for...of vs for...in [F-41]

⚠ Diferencia crítica en TypeScript:

Forma	Itera sobre	Resultado
<code>for...of</code>	Valores del iterable	<code>10, 20, 30</code>
<code>for...in</code>	Claves del objeto (strings)	<code>"0", "1", "2"</code>

```
const arr = [10, 20, 30]  
for (const k in arr) console.log(k) // "0", "1", "2" → claves  
for (const v of arr) console.log(v) // 10, 20, 30 → valores correctos
```

Regla: para *arrays*, siempre `for...of`. `for...in` es para enumerar propiedades de objetos.

4.4.9 Recursión vs. iteración [F-42]

Misma potencia expresiva, costos operacionales distintos:

- **Recursión:** estilo declarativo; el estado está implícito en la **pila de llamadas**.
- **Iteración:** control operativo explícito; el estado está en **variables locales**.
- Toda recursión primitiva puede transformarse en iteración con una pila explícita.
- Equivalencia expresiva con costos operacionales distintos (stack vs. heap).

Tail call optimization (TCO): Haskell, Scheme y Scala optimizan recursión en cola como iteración — elimina el costo de stack. ⚠️ TypeScript **no** tiene TCO: la recursión profunda puede agotar el stack.

“In general, it is always possible to transform a function definition which is not tail recursive into an equivalent tail recursive one, by complicating the definition. The idea is that all the computations which have to be made after the recursive call (...) should be performed before the call itself.” — [Gabbrielli & Martini, *Programming Languages: Principles and Paradigms*, cap. 6, p. 136]

“Recursion of the kind used in the function `factrc` is said to be tail recursion, since the recursive call is, so to speak, the last thing that happens in the body of the procedure. After the recursive call, no other computation is performed.” — [Gabbrielli & Martini, *Programming Languages: Principles and Paradigms*, cap. 6, p. 136]

“However, many functions that use recursion for repetition are not tail recursive. Programmers who are concerned with efficiency have discovered ways to rewrite some of these functions so that they are tail recursive. One example of this uses an accumulating parameter and a helper function.” — [Sebesta, *Concepts of Programming Languages*, cap. 9 (Subprogramas), p. 647]

4.5 Bloque E — Integración y cierre [F-43 a F-44]

4.5.1 Control asíncrono con `async/await` [F-43]

`async/await` ofrece **secuencialidad aparente** sobre operaciones asíncronas.

- `await` **suspende** la función actual sin bloquear el hilo de ejecución.
- Permite escribir flujo de control lineal sobre operaciones inherentemente concurrentes.
- El *runtime* convierte el código en una **máquina de estados implícita**.

```
async function obtenerDatos(): Promise<string[]> {  
  return ["A", "B", "C"];  
}  
  
async function pipeline() {  
  const datos    = await obtenerDatos()    // se suspende hasta resolución  
  const validado = await validar(datos)  
  const resultado = await transformar(validado)  
  return resultado  
}
```

El código se lee como secuencial, pero el *runtime* lo transforma en una máquina de estados. Es la estructura de control más moderna que vimos y prepara el terreno para el módulo XI (conurrencia).

⚠️ `await` **no bloquea el hilo**: suspende la función sin bloquear el hilo. El hilo puede hacer otra cosa mientras espera.

4.5.2 Cierre: cinco ideas fuerza [F-44]

1. **Parseo ≠ evaluación:** la precedencia fija la forma; el orden de evaluación lo fija el lenguaje.
2. **Short-circuit por corrección:** la semántica de corto circuito evita estados inválidos, no solo optimiza.
3. **Estructuras por mantenibilidad:** if-else, switch o dispatch según el dominio y el cambio esperado.
4. **Iteración correcta:** un bucle sin invariante y sin función de ranking no es correcto por accidente.
5. **Legibilidad semántica:** nombrar bien predicados y estructurar el flujo previene defectos futuros.

5. Ejemplos trabajados

Ejemplo A — Precedencia, asociatividad y orden de evaluación

Enunciado: Analizá la expresión `let i = 1; const r = i++ + ++i` en TypeScript. ¿Qué valor tiene `r`? ¿Por qué? ¿Qué cambia en C y en Rust?

Resolución paso a paso:

1. **Parseo:** el AST agrupa como `(i++) + (++i)`. Ambos `+` son el mismo operador binario; la asociatividad no resuelve nada aquí porque hay un solo `+`. La precedencia no crea ambigüedad estructural.
2. **Orden de evaluación:** TypeScript evalúa los operandos de `+` de izquierda a derecha.
 - Primero se evalúa `i++`: devuelve el valor actual de `i` (1) y luego incrementa `i` a 2.
 - Después se evalúa `++i`: incrementa `i` de 2 a 3 y devuelve 3.
 - La suma: `1 + 3 = 4`. Entonces `r = 4` e `i = 3`.
3. **Contraste con C:** en C/C++, la expresión `i = i++ + ++i` es **comportamiento indefinido** — el estándar no garantiza un orden, y el compilador puede producir cualquier resultado. No es un bug del compilador; es una expresión ilegal según el estándar.
4. **Contraste con Rust:** Rust **no tiene** `i++` ni `++i`. Solo existe `i += 1` (sentencia, no expresión). La decisión de diseño elimina la clase entera de bugs.

Conclusión: aunque TypeScript define el orden, la expresión es **ilegible** — un humano no puede razonar sobre ella sin conocer la tabla de orden del lenguaje. Definido ≠ correcto. La regla de diseño es: evitar expresiones que mezclen cálculo y mutación en la misma sentencia.

Fundamento: [Sebesta, cap. 7, §7.2, p. 325] — “Operand evaluation order is important if functions have side effects.”

Ejemplo B — Short-circuit defensivo

Enunciado: Dado el siguiente código, explicá por qué funciona y qué pasa si se invierte el orden de las condiciones.

```
if (x !== 0 && y / x > 2) {  
  aprobar()  
}
```

Resolución paso a paso:

1. **Semántica de && con short-circuit:** $p \ \&\& \ q$ evalúa p primero. Si p es `false`, el resultado ya es `false` y q **no se evalúa**. Si p es `true`, recién entonces se evalúa q .
2. **Caso $x = 0$:** la primera condición $x \neq 0$ es `false`. El short-circuit detiene la evaluación: $y / x > 2$ **nunca se ejecuta**. No hay división por cero. El `if` no entra y `aprobar()` no se llama.
3. **Caso $x = 5, y = 15$:** la primera condición $x \neq 0$ es `true`. Se evalúa la segunda: $y / x > 2 \rightarrow 15 / 5 > 2 \rightarrow 3 > 2 \rightarrow \text{true}$. El `if` entra y `aprobar()` se llama.
4. **Inversión del orden (anti-patrón):** `if (y / x > 2 && x !== 0)`. Cuando $x = 0$, la primera condición $y / x > 2$ se evalúa **antes** de la guarda $\rightarrow$ división por cero $\rightarrow$ `NaN` (TypeScript) o *runtime error* (otros lenguajes). La guarda se destruye.

Conclusión: el orden de las condiciones en un `&&` con short-circuit es **semántico, no estético**. La primera condición actúa como guarda operacional de la segunda.

Fundamento: [Gabbrielli & Martini, cap. 6, p. 136] — “The main reason for a short-circuit evaluation of Boolean operators is not efficiency (...). With full evaluation, certain common program schemata would be incorrect.”

Ejemplo C — Invariante de bucle

Enunciado: Dado el siguiente bucle que suma los primeros n naturales, formulá el invariante y demostrá que se preserva.

```
let suma = 0  
let i = 0  
while (i < n) {  
  suma = suma + i  
  i = i + 1  
}  
// Al terminar: suma = 0 + 1 + 2 + ... + (n-1)
```

Resolución paso a paso:

1. **Invariante propuesto:** $\text{suma} = 0 + 1 + 2 + \dots + (i - 1)$ (la suma de los primeros i naturales, empezando desde 0). Equivalentemente: “suma contiene la suma de todos los enteros desde 0 hasta $i - 1$ ”.
2. **Establecimiento (antes de entrar):** cuando $i = 0$ y $\text{suma} = 0$, el invariante dice “suma = suma de los enteros desde 0 hasta -1 ” = suma vacía = 0. ✓ El invariante se establece.
3. **Preservación (en cada iteración):** supongamos que el invariante vale antes de una iteración: $\text{suma} = 0 + 1 + \dots + (i - 1)$. El cuerpo ejecuta:
 - $\text{suma} = \text{suma} + i \rightarrow$ ahora $\text{suma} = 0 + 1 + \dots + (i - 1) + i = 0 + 1 + \dots + i$.
 - $i = i + 1 \rightarrow$ ahora i vale $i + 1$.
 - El invariante después de la iteración: “suma = $0 + 1 + \dots + (i_{\text{nuevo}} - 1) = 0 + 1 + \dots + ((i+1) - 1) = 0 + 1 + \dots + i$ ”. ✓ Coincide.
4. **Conclusión al terminar:** el bucle termina cuando $i < n$ es falso, es decir, $i \geq n$. Combinando el invariante ($\text{suma} = 0 + 1 + \dots + (i - 1)$) con la negación de la condición ($i \geq n$), y dado que i se incrementa de a 1 desde 0, al terminar $i = n$. Por lo tanto: $\text{suma} = 0 + 1 + \dots + (n - 1)$. ✓ Resultado correcto.
5. **Loop variant (terminación):** la expresión $n - i$ es un entero no negativo que decrece en cada iteración (porque i aumenta en 1). Cuando $n - i = 0$, el bucle termina. El variant es estrictamente decreciente y acotado inferiormente por 0. ✓ Terminación garantizada.

Conclusión: el invariante + el variant demuestran corrección total (parcial + terminación). Un bucle sin invariante y sin función de ranking no es correcto por accidente.

Fundamento: [Sebesta, cap. 3 (Lógica de Hoare), p. 25] — “The loop invariant must satisfy a number of requirements to be useful.”

Ejemplo D — Conversión de iteración a recursión

Enunciado: Transformá la función iterativa `factorial` en una versión recursiva, y luego en una versión con recursión en cola (*tail recursive*). Explicá la diferencia.

Versión iterativa:

```
function factorialIter(n: number): number {
  let resultado = 1
  for (let i = 1; i <= n; i++) {
    resultado = resultado * i
  }
  return resultado
}
```

Versión recursiva (no tail recursive):

```
function factorialRec(n: number): number {
  if (n <= 1) return 1
  return n * factorialRec(n - 1) // la multiplicación queda PENDIENTE después de la llamada
}
```

El estado está implícito en la pila de llamadas: cada llamada espera el resultado de la siguiente para multiplicar. La recursión **no es en cola**: después de la llamada recursiva queda la operación `n * ...` pendiente.

Versión tail recursive (con parámetro acumulador):

```
function factorialTail(n: number, acc: number = 1): number {
  if (n <= 1) return acc
  return factorialTail(n - 1, n * acc) // la llamada recursiva es lo ÚLTIMO que pasa
}
```

Aquí la llamada recursiva es lo último que ocurre: no queda ninguna operación pendiente. El acumulador `acc` lleva el resultado parcial. En un lenguaje con **TCO** (Haskell, Scheme, Scala), el compilador transforma esto en un bucle equivalente sin consumir stack.

Análisis comparativo:

Aspecto	Iterativa	Recursiva (no tail)	Tail recursive
Estado	Variables locales	Pila de llamadas	Parámetro acumulador
Costo de stack	O(1)	O(n)	O(1) con TCO / O(n) sin TCO
Estilo	Operativo	Declarativo	Declarativo + eficiente
TypeScript	Óptimo	Riesgo de stack overflow	Riesgo de stack overflow (sin TCO)

Conclusión: toda recursión primitiva puede transformarse en iteración con una pila explícita. La equivalencia es expresiva, pero los costos operacionales difieren (stack vs. heap). En TypeScript, que no tiene TCO, la versión iterativa es la más segura para `n` grande.

Fundamento: [Gabbrielli & Martini, cap. 6, p. 136] — “It is always possible to transform a function definition which is not tail recursive into an equivalent tail recursive one.” [Sebesta, cap. 9, p. 647] — “Programmers who are concerned with efficiency have discovered ways to rewrite some of these functions so that they are tail recursive. One example of this uses an accumulating parameter and a helper function.”

6. Puntos clave (cheat-sheet)

Expresión vs. sentencia

- **Expresión:** produce valor. **Sentencia:** causa efecto o controla ejecución.
- En TypeScript, la asignación es ambas cosas. En Kotlin, solo sentencia.

Precedencia y asociatividad

- **Precedencia:** qué operador agrupa primero (más profundo en el AST).
- **Asociatividad:** cómo se resuelven empates (izquierda: `+` `-` `*` `/` ; derecha: `**` , asignación, ternario).
- No es universal: varía entre lenguajes. Cuando hay duda, usar paréntesis.

Orden de evaluación

- **Parseo ≠ evaluación temporal.** El AST fija la forma; el lenguaje fija el orden temporal.
- Con funciones puras no importa. Con efectos colaterales, importa mucho.
- C/C++: casos indefinidos. TypeScript: más definido pero ilegible. Rust: elimina `i++` .

Side effect

- Cualquier cambio de estado fuera de la expresión. Rompe transparencia referencial.
- Separar predicados (sin efectos) de funciones con efectos.

Short-circuit

- `&&` : si el primer operando es false, no evalúa el segundo.
- `||` : si el primer operando es true, no evalúa el segundo.
- Es corrección semántica, no solo optimización. El orden de condiciones es semántico.
- Operadores bit a bit (`&` , `|`) **no** cortocircuitan.

Truthiness

- Qué cuenta como `true` varía: C usa `int` , Python/TypeScript son flexibles, Java/Kotlin son estrictos.
- No migrar intuiciones entre lenguajes sin validar.

Estructuras de selección

- **if-else:** pocas ramas o condiciones complejas.
- **switch/when/match:** muchas ramas por valor; verificar exhaustividad con `default` / `else` .
- **Tabla de despacho:** cambio frecuente de casos; principio Open/Closed.
- Regla: máximo 3 niveles de anidamiento antes de extraer una función.

Iteración con invariantes

- **Invariante:** propiedad que se mantiene verdadera antes, durante y después del bucle.
- **Loop variant:** función de ranking que decrece y garantiza terminación.
- Un bucle sin invariante y sin variant no es correcto por accidente.

break / continue

- Transferencias **estructuradas**: solo afectan el bucle más cercano.
- No son `goto`: no saltan a cualquier punto.
- Java permite `break label` para bucles anidados.

Recursión vs. iteración

- Misma potencia expresiva, costos distintos (stack vs. heap).
- **Tail call optimization (TCO)**: Haskell, Scheme, Scala. TypeScript no lo tiene.
- Toda recursión primitiva → iteración con pila explícita.

async/await

- `await` suspende la función sin bloquear el hilo.
- El runtime lo convierte en una máquina de estados implícita.
- Secuencialidad aparente sobre concurrencia real.

7. Autoevaluación

12 preguntas distribuidas según la taxonomía de Bloom. Las respuestas están colapsadas en `<details>` — intentá responder antes de abrir.

#	Pregunta	Nivel Bloom
1	¿Cuál es la diferencia entre precedencia de operadores y orden de evaluación de operandos?	Comprender
2	Definí "expresión" y "sentencia" con tus palabras y dà un ejemplo de cada una.	Recordar
3	Dada <code>const r = 10 - 3 - 2</code> , ¿qué valor da y por qué?	Aplicar
4	¿Cuándo una <i>assignment expression</i> en un <code>if</code> se convierte en bug?	Analizar
5	Da un ejemplo donde la coerción implícita produce un resultado diferente al esperado.	Aplicar
6	Explicá por qué <code>if (x !== 0 && y / x > 2)</code> es seguro pero <code>if (y / x > 2 && x !== 0)</code> no lo es.	Analizar
7	¿Por qué el short-circuit es una herramienta de corrección semántica y no solo de rendimiento?	Evaluar
8	Dado un bucle que suma los primeros <code>n</code> enteros, formulá el invariante y el loop variant.	Crear
9	Compará <code>switch</code> con <code>when</code> de Kotlin: ¿qué problema de confiabilidad elimina <code>when</code> y cómo?	Analizar
10	¿Qué es la recursión en cola y por qué importa para la terminación de stack?	Comprender
11	Justificá cuándo usarías una tabla de despacho en lugar de un <code>switch</code> .	Evaluar
12	Escribí un generador en TypeScript que produzca los números pares del 0 al infinito bajo demanda.	Crear

Respuestas

- ▶ Respuesta 1 (Comprender)
- ▶ Respuesta 2 (Recordar)
- ▶ Respuesta 3 (Aplicar)
- ▶ Respuesta 4 (Analizar)
- ▶ Respuesta 5 (Aplicar)
- ▶ Respuesta 6 (Analizar)
- ▶ Respuesta 7 (Evaluar)
- ▶ Respuesta 8 (Crear)
- ▶ Respuesta 9 (Analizar)
- ▶ Respuesta 10 (Comprender)
- ▶ Respuesta 11 (Evaluar)
- ▶ Respuesta 12 (Crear)

8. Glosario

Término	Definición
Expresión	Construcción sintáctica que se evalúa y produce un valor. Ej: <code>x + 1</code> .
Sentencia	Unidad ejecutable del programa que controla la ejecución o provoca un efecto. Ej: <code>let x = 5</code> .
AST	Árbol Sintáctico Abstracto (<i>Abstract Syntax Tree</i>): representación intermedia que captura la estructura de una expresión según la precedencia y asociatividad. No captura el orden temporal de evaluación.
Precedencia	Regla que determina qué operador agrupa primero en una expresión. El operador con mayor precedencia queda más profundo en el AST. Es decisión de diseño del lenguaje.
Asociatividad	Regla que resuelve empates entre operadores de igual precedencia. Izquierda: <code>+ - * /</code> . Derecha: <code>**</code> , asignación en C/Java, ternario.
Orden de evaluación	Orden temporal en que se evalúan los operandos durante la ejecución. Lo define el lenguaje, no el AST. Con efectos colaterales, importa mucho.
Side effect (efecto colateral)	Cualquier cambio de estado fuera de la expresión que se está evaluando (modificación de variables, I/O, mutación). Rompe la transparencia referencial.
Short-circuit	Semántica de evaluación perezosa de operadores lógicos: <code>&&</code> no evalúa el segundo operando si el primero es false; `
Truthiness	Convención de un lenguaje sobre qué valores no booleanos cuentan como <code>true</code> o <code>false</code> en una condición. Varía: C usa <code>int</code> , Python/TypeScript son flexibles, Java/Kotlin son estrictos.
if/else	Estructura de selección que divide el flujo en ramas mutuamente excluyentes, evaluadas en orden. En Kotlin, <code>if</code> es una expresión que devuelve valor.
switch	Estructura de selección múltiple que discrimina por valor. En lenguajes basados en C requiere <code>break</code> para evitar fallthrough. Kotlin lo reemplaza por <code>when</code> ; Rust por <code>match</code> .
Pattern matching	Evolución del <code>switch</code> que discrimina por estructura del dato (forma, tipo, desestructuración), no solo por valor. Disponible en Haskell, Scala, Rust, Python 3.10+, Java 21+.
while / for	Estructuras iterativas. <code>while</code> : evalúa condición al inicio. <code>do-while</code> : al final (garantiza una ejecución). <code>for</code> : contador + condición + actualización en una línea; azúcar sintáctico sobre <code>while</code> en la mayoría de lenguajes modernos.
Loop invariant (invariante de bucle)	Propiedad que se mantiene verdadera antes, durante y después de cada iteración del bucle. Base de la verificación formal (Lógica de Hoare). Permite demostrar corrección parcial.

Término	Definición
break	Transferencia estructurada que sale del bucle más cercano. No es <code>goto</code> : solo afecta el bucle que la contiene. Java permite <code>break label</code> para bucles anidados.
continue	Transferencia estructurada que salta a la próxima iteración del bucle más cercano, omitiendo el resto del cuerpo actual.
Generator (generador)	Función que produce valores bajo demanda usando <code>yield</code> . Suspende la ejecución en cada <code>yield</code> y la reanuda al siguiente <code>next()</code> . Permite secuencias infinitas sin memoria proporcional.
yield	Palabra clave que suspende la ejecución de un generador y devuelve un valor. El estado (activación, posición, memoria) se preserva para la próxima reanudación.
Recursion (recursión)	Técnica donde una función se llama a sí misma. El estado está implícito en la pila de llamadas. Estilo declarativo.
Tail call (llamada en cola)	Llamada recursiva que es la última operación del cuerpo de la función: después de ella no queda computación pendiente. Permite TCO: el compilador la transforma en iteración sin costo de stack.
async/await	Estructura de control moderno para concurrencia. <code>await</code> suspende la función sin bloquear el hilo. El runtime convierte el código en una máquina de estados implícita. Ofrece secuencialidad aparente sobre operaciones asíncronas.
Coerción	Conversión de tipo implícita realizada por el lenguaje. <i>Widening</i> (sin pérdida, riesgo medio) o <i>narrowing</i> (con pérdida, riesgo alto). Puede enmascarar errores de tipo.
Guard clause	Cláusula de guarda que retorna temprano de una función si una precondition no se cumple. Reduce anidamiento y complejidad accidental sin cambiar la semántica.

9. Referencias y lecturas recomendadas

Bibliografía principal

1. **Sebesta, R. W.** (2019). *Concepts of Programming Languages* (12.^a ed.). Pearson.
 - **Cap. 7 — Expressions and Assignment Statements** (pp. 325–388): expresiones aritméticas, precedencia y asociatividad (§7.2), orden de evaluación de operandos, operadores sobrecargados (§7.3), conversiones de tipo (§7.4), expresiones relacionales y booleanas (§7.5), short-circuit evaluation (§7.6), sentencias de asignación (§7.7), mixed-mode assignment (§7.8).
 - **Cap. 8 — Statement-Level Control Structures:** selección (if-else, switch), iteración (while, do-while, for), transferencia incondicional (goto), debate sobre programación estructurada, guarded commands de Dijkstra.
 - **Cap. 3 — Verificación de programas** (pp. 25–184): Lógica de Hoare, invariantes de bucle, weakest preconditions.
 - **Cap. 9 — Subprogramas** (pp. 647–702): recursión, tail recursion, parámetros acumuladores.
 - **Bibliografía** (pp. 703–784): Dijkstra, E. W. (1968a). "Goto Statement Considered Harmful." *Commun. ACM*, 11(3), 147–149.
2. **Gabbrielli, M. & Martini, S.** (2023). *Programming Languages: Principles and Paradigms* (2.^a ed.). Springer (UTCS).
 - **Cap. 6 — Control Structures** (pp. 136–282): semántica operacional de expresiones, evaluación estricta vs. short-circuit, asignación, estructuras iterativas, invariantes de bucle, recursión y tail call optimization, generadores con `yield`, programación estructurada y debate sobre goto (§6.4).
3. **Louden, K. C. & Lambert, K. A.** (2012). *Programming Languages: Principles and Practice* (3.^a ed.). Cengage Learning.
 - **Cap. 8:** implementación del control de flujo, legibilidad y decisiones de diseño de lenguajes, truthiness y valores de verdad.

Referencias históricas

- **Dijkstra, E. W.** (1968). "Go To Statement Considered Harmful." *Communications of the ACM*, Vol. 11, No. 3, pp. 147–149. Referenciado en [Sebesta, bibliografía, p. 703].
- **Dijkstra, E. W.** (1975). Guarded commands, nondeterminacy and formal derivation of programs. *Communications of the ACM*, 18(8), 453–457. Referenciado en [Sebesta, cap. 8, p. 557].

Lecturas recomendadas para profundizar

- Para **verificación formal e invariantes**: Sebesta cap. 3 (Lógica de Hoare, weakest preconditions).
 - Para **semántica operacional de short-circuit y coerciones**: Gabbrielli & Martini cap. 6.
 - Para **tail call optimization y transformación recursión** → **iteración**: Gabbrielli & Martini cap. 6 (continuation passing style).
 - Para **implementación del control de flujo y legibilidad**: Louden cap. 8.
 - Para **pattern matching y tipos algebraicos**: documentación de Rust (`match`), Haskell, Scala.
-

Guía generada por Dra. Sofía (study-guide-writer) el 2026-06-28. Baseline de contenido:

`clase_dada.txt` (859 líneas), `filminas.md` (F-00 a F-44), `minuta.md` (180 min), `diseno.md` (corregido). Citas verificadas en ChromaDB (knowledge base material): Sebesta 2019, Gabbrielli & Martini 2023. Scope definido por `diseno.md` — no incluye contenido fuera del diseño aprobado. “Si un alumno puede estudiarlo solo, lo hicimos bien.”